

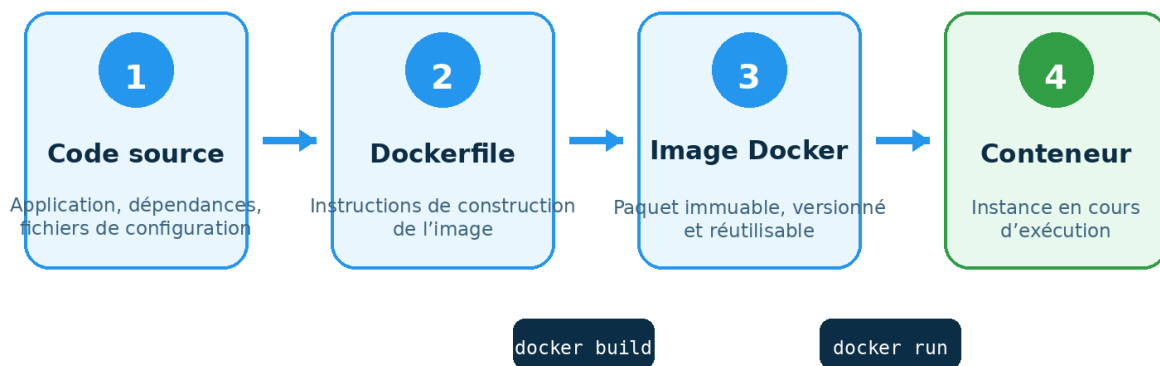
DOCKER

Images, conteneurs et construction d'une application

Guide débutant illustré avec un exemple Spring Boot

Le cycle Docker : du code au conteneur

Docker transforme un projet en environnement portable et exécutable.



Idée essentielle Dockerfile construit une image ; l'image permet de créer un ou plusieurs conteneurs.

Objectif : comprendre ce qu'est Docker, pourquoi l'utiliser, la différence entre une image et un conteneur, puis construire et lancer sa première image.

Sommaire

1	Qu'est-ce que Docker ?
2	Pourquoi utiliser Docker ?
3	Les composants principaux
4	Image Docker et conteneur
5	Le Dockerfile
6	Construire une image étape par étape
7	Exemple Spring Boot
8	Commandes essentielles
9	Docker Compose et architecture multi-conteneurs
10	Bonnes pratiques et résumé

Lecture recommandée : commencer par les sections 1 à 6, puis utiliser l'exemple Spring Boot comme exercice pratique.

1. Qu'est-ce que Docker ?

Docker est une plateforme de conteneurisation. Elle permet d'empaqueter une application avec les éléments nécessaires à son exécution — code, runtime, bibliothèques, dépendances et configuration — dans une image portable.

À partir de cette image, Docker démarre un conteneur : un processus isolé qui exécute l'application de manière prévisible sur un ordinateur de développement, un serveur, une plateforme cloud ou une pipeline CI/CD.

Phrase simple : Docker sert à exécuter la même application de la même manière dans différents environnements.

2. Pourquoi utiliser Docker ?

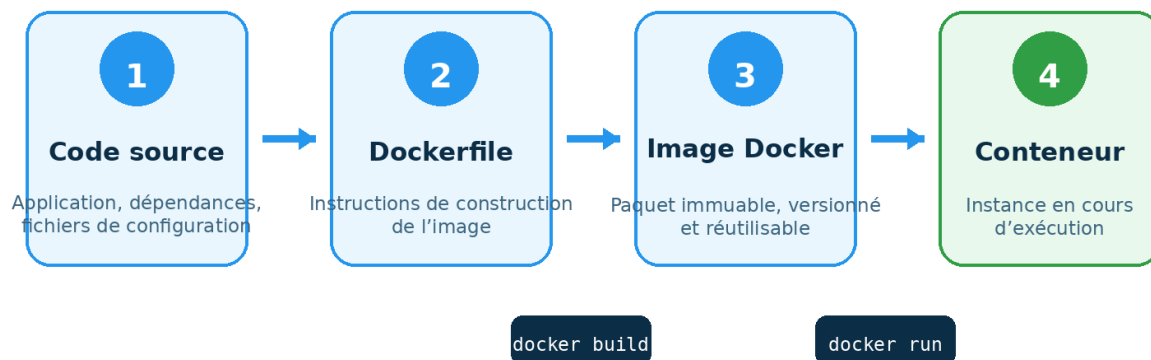
Avantage	Explication
Portabilité	Le même paquet peut être utilisé sur plusieurs machines compatibles Docker.
Cohérence	Réduit le problème « ça fonctionne sur mon ordinateur, mais pas sur le serveur ».
Isolation	Chaque conteneur possède son propre environnement de processus et de fichiers.
Déploiement rapide	Une image déjà construite peut être démarrée rapidement.
Versionnement	Les images peuvent avoir des tags comme 1.0, 1.1 ou latest.
CI/CD	La même image testée peut être publiée puis déployée automatiquement.
Architecture modulaire	Frontend, backend et base de données peuvent être séparés en services.

3. Les composants principaux de Docker

Dockerfile Fichier texte qui décrit comment construire une image.	Image Paquet immuable contenant l'environnement de l'application.	Conteneur Instance en cours d'exécution créée à partir d'une image.
Docker Engine Moteur qui construit les images et gère les conteneurs.	Registry Dépôt d'images, par exemple Docker Hub ou un registre privé.	Docker Compose Outil de description et de démarrage d'une application composée de plusieurs services.

Le cycle Docker : du code au conteneur

Docker transforme un projet en environnement portable et exécutable.



Idée essentielle Dockerfile construit une image ; l'image permet de créer un ou plusieurs conteneurs.

Figure 1 — Relation entre le code, le Dockerfile, l'image et le conteneur.

4. Image Docker et conteneur

4.1 Qu'est-ce qu'une image Docker ?

Une image Docker est un paquet standardisé et immuable. Elle contient les fichiers, binaires, bibliothèques et configurations nécessaires au démarrage d'un conteneur. Une image peut être téléchargée depuis un registry ou construite localement avec un Dockerfile.

- Elle n'est pas un programme actif : elle est stockée sur le disque.
- Elle est en lecture seule après sa construction.
- Elle peut porter un nom et un tag : `matchia-backend:1.0`.
- Une même image peut créer plusieurs conteneurs indépendants.

4.2 Qu'est-ce qu'un conteneur ?

Un conteneur est une instance exécutable d'une image. Il possède un état d'exécution, un nom, des ports, des variables d'environnement et éventuellement des volumes pour conserver les données.

- Il peut être démarré, arrêté, redémarré ou supprimé.
- Il exécute généralement un processus principal.
- Ses modifications temporaires disparaissent lorsqu'il est supprimé, sauf si des volumes sont utilisés.
- Plusieurs conteneurs basés sur la même image peuvent fonctionner simultanément.

Image Docker et conteneur : quelle différence ?

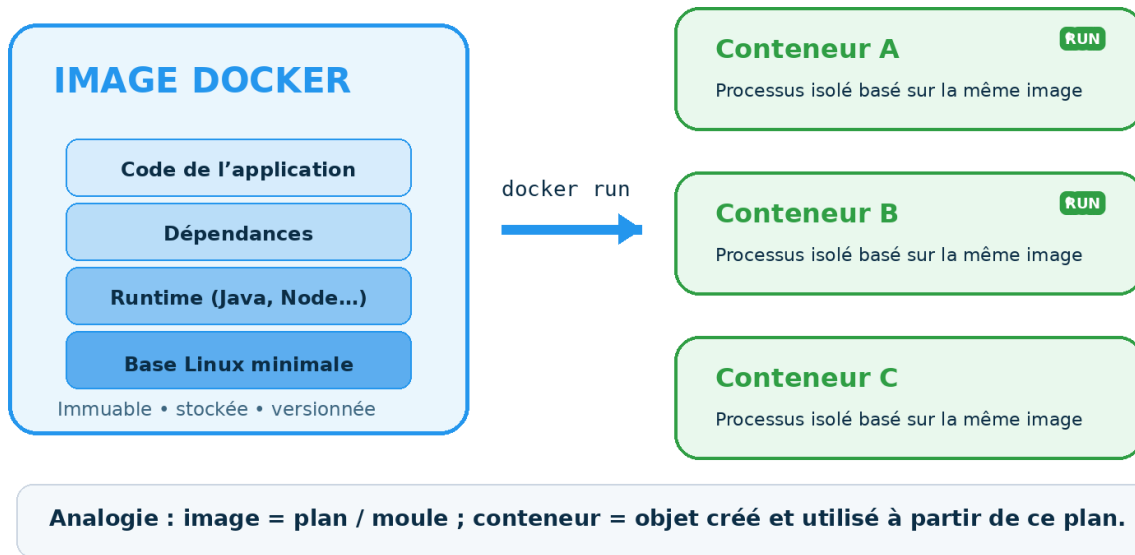


Figure 2 — Une image peut servir de modèle à plusieurs conteneurs.

4.3 Tableau comparatif

Critère	Image	Conteneur
Nature	Modèle immuable	Instance en exécution
Création	docker build ou docker pull	docker run / docker create
État	Stockée	Démarrée, arrêtée ou supprimée
Modification	Nouvelle image à construire	État temporaire possible
Exemple	postgres:18	Conteneur PostgreSQL de développement

5. Le Dockerfile

Un **Dockerfile** est un fichier texte nommé généralement **Dockerfile**, sans extension. Docker lit ses instructions dans l'ordre afin de construire l'image.

5.1 Instructions principales

Instruction	Rôle
FROM	Choisit l'image de base et démarre une étape de construction.
WORKDIR	Définit le dossier de travail dans l'image.
COPY	Copie les fichiers du contexte de build vers l'image.
RUN	Exécute une commande pendant la construction.
ENV	Définit une variable d'environnement.
EXPOSE	Documente le port écouté par l'application.
CMD	Définit la commande par défaut au démarrage.
ENTRYPOINT	Définit l'exécutable principal du conteneur.

5.2 Les couches d'une image

Une image est constituée de couches. Lorsqu'une instruction modifie le système de fichiers, Docker peut créer une nouvelle couche. Les couches inchangées peuvent être récupérées depuis le cache, ce qui accélère les constructions suivantes.

Une image Docker est composée de couches

Chaque instruction importante du Dockerfile peut produire une couche réutilisable.

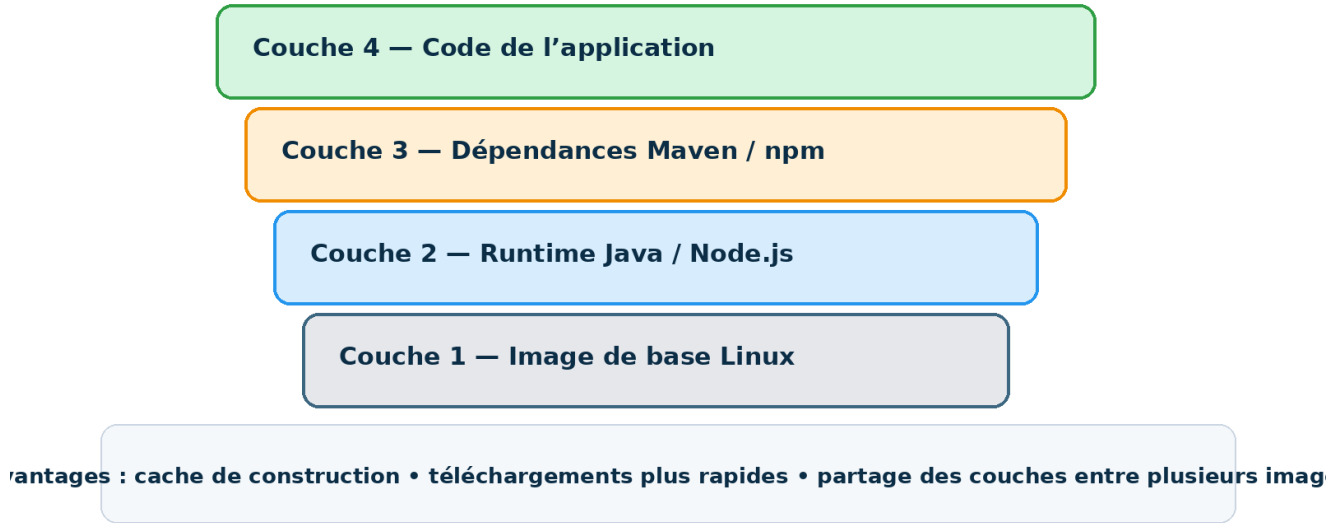


Figure 3 — Exemple simplifié des couches d'une image applicative.

Attention : EXPOSE ne publie pas automatiquement un port sur la machine hôte. La publication se fait avec `-p` lors de `docker run` ou avec `ports` dans Docker Compose.

6. Construire une image étape par étape

6.1 Organisation minimale du projet

```
mon-projet/  
├── Dockerfile  
├── .dockerignore  
├── fichier-dependances  
└── src/
```

Le dernier argument de `docker build` représente le contexte de construction. Avec la valeur « `.` », Docker peut accéder aux fichiers du dossier courant et de ses sous-dossiers, sauf ceux exclus par `.dockerignore`.

6.2 Créer un Dockerfile

```
FROM image-de-base:version  
WORKDIR /app  
COPY . .  
RUN commande-de-construction  
EXPOSE 8080  
CMD ["commande", "argument"]
```

6.3 Construire l'image

```
docker build -t mon-application:1.0 .
```

- `docker build` : demande la construction d'une image.
- `-t mon-application:1.0` : attribue un nom et un tag.
- `.` : utilise le dossier courant comme contexte de build.

6.4 Lancer un conteneur

```
docker run -d -p 8080:8080 --name mon-conteneur mon-application:1.0
```

- `-d` : exécution en arrière-plan.
- `-p 8080:8080` : associe le port 8080 de la machine au port 8080 du conteneur.
- `--name` : donne un nom lisible au conteneur.
- `mon-application:1.0` : image utilisée.

Construire et lancer une image : les commandes

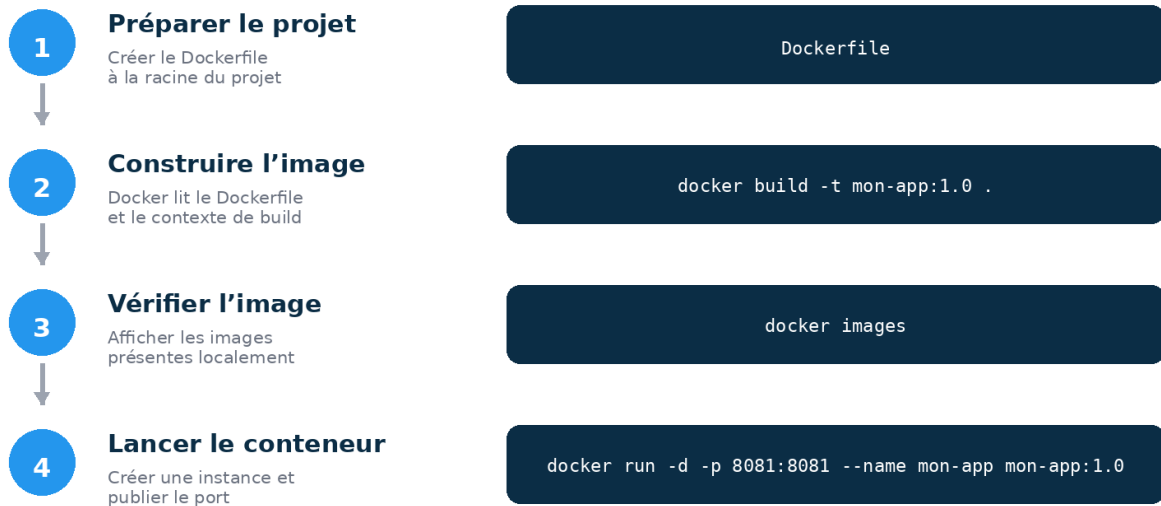


Figure 4 — Séquence de construction et de lancement.

7. Exemple pratique : backend Spring Boot

Cet exemple suppose que le projet utilise Java 17, Maven et que l'application écoute sur le port 8081.

7.1 Construire le fichier JAR

```
mvn clean package -DskipTests
```

Après cette commande, Maven produit un fichier JAR dans le dossier target/. Vérifie son nom avant d'écrire la commande COPY.

7.2 Dockerfile simple

```
FROM eclipse-temurin:17-jre
WORKDIR /app
COPY target/matchia-backend.jar app.jar
EXPOSE 8081
ENTRYPOINT ["java", "-jar", "app.jar"]
```

7.3 Fichier .dockerignore

```
.git
.idea
.vscode
*.log
node_modules
README.md
```

Le fichier .dockerignore évite d'envoyer des fichiers inutiles dans le contexte de build. Le dossier target ne doit pas être ignoré dans cet exemple, car le JAR doit être copié dans l'image.

7.4 Construire et lancer

```
docker build -t matchia-backend:1.0 .
docker run -d -p 8081:8081 --name matchia-backend matchia-backend:1.0
```

7.5 Vérifier le résultat

```
docker ps
docker logs -f matchia-backend
docker inspect matchia-backend
```

Cas fréquent : si le nom réel du JAR change à chaque version, utilise un nom de fichier Maven stable ou adapte la ligne COPY.

8. Commandes Docker essentielles

Commande	Utilité
<code>docker version</code>	Afficher les versions client et serveur.
<code>docker info</code>	Afficher l'état général du moteur Docker.
<code>docker pull nginx</code>	Télécharger une image.
<code>docker images</code>	Lister les images locales.
<code>docker build -t app:1.0 .</code>	Construire une image.
<code>docker run IMAGE</code>	Créer et démarrer un conteneur.
<code>docker ps</code>	Lister les conteneurs actifs.
<code>docker ps -a</code>	Lister tous les conteneurs.
<code>docker logs -f NOM</code>	Suivre les journaux.
<code>docker exec -it NOM sh</code>	Ouvrir un shell dans un conteneur compatible.
<code>docker stop NOM</code>	Arrêter un conteneur.
<code>docker start NOM</code>	Redémarrer un conteneur arrêté.
<code>docker rm NOM</code>	Supprimer un conteneur arrêté.
<code>docker rmi IMAGE</code>	Supprimer une image inutilisée.
<code>docker system df</code>	Afficher l'espace utilisé par Docker.

Sécurité : évite de placer des mots de passe, clés API ou secrets directement dans un Dockerfile ou dans une image. Utilise des variables d'environnement, des fichiers de secrets non versionnés ou le gestionnaire de secrets de la plateforme.

9. Docker Compose et les applications multi-conteneurs

Une plateforme comme Matchia peut comporter plusieurs services : frontend React, backend Spring Boot et base PostgreSQL. Il est préférable de créer un conteneur par service, puis de décrire leurs relations dans un fichier `compose.yaml`.

Exemple d'une application complète avec Docker Compose

Chaque service possède son propre conteneur et communique sur un réseau Docker.

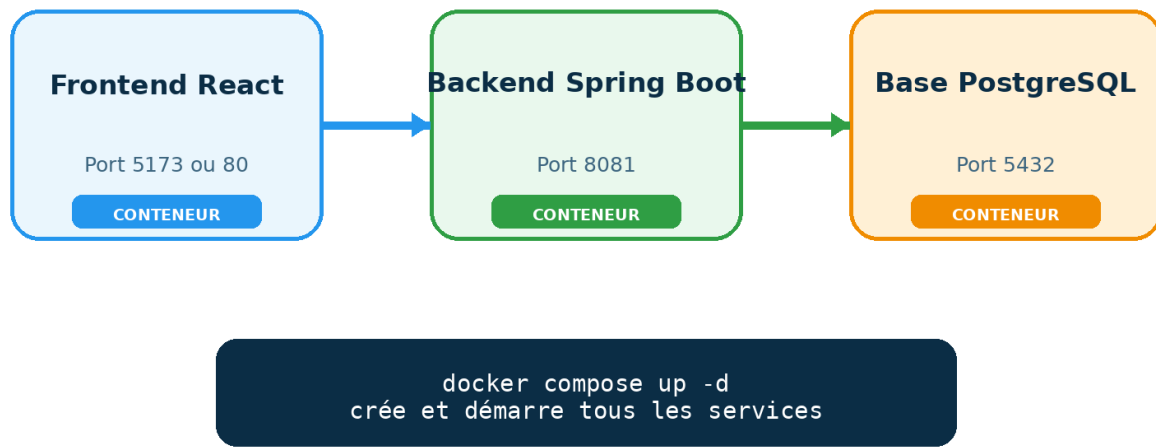


Figure 5 — Exemple de communication entre trois conteneurs.

9.1 Exemple simplifié de compose.yaml

```

services:
  backend:
    build: ./backend
    ports:
      - "8081:8081"
    environment:
      SPRING_DATASOURCE_URL: jdbc:postgresql://database:5432/matchia
    depends_on:
      - database

  frontend:
    build: ./frontend
    ports:
      - "5173:80"
    depends_on:
      - backend

  database:
    image: postgres:18
    environment:
      POSTGRES_DB: matchia
      POSTGRES_USER: matchia
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
    volumes:
      - postgres_data:/var/lib/postgresql/data

volumes:
  postgres_data:
  
```

```
docker compose up -d --build
```

Point important : dans le réseau Compose, le backend contacte PostgreSQL avec le nom du service « database », et non avec localhost.

10. Bonnes pratiques

- ✓ **Choisir une image de base adaptée** — Utiliser une version précise et une image officielle lorsque cela convient.
- ✓ **Réduire la taille** — Ne copier que les fichiers nécessaires et utiliser `.dockerignore`.
- ✓ **Exploiter le cache** — Copier les fichiers de dépendances avant le code qui change souvent.
- ✓ **Ne pas exécuter en root** — Créer un utilisateur applicatif lorsque l'image le permet.
- ✓ **Ne pas stocker les secrets** — Injecter les secrets au lancement ou depuis la plateforme de déploiement.
- ✓ **Utiliser des volumes** — Conserver les données de base en dehors du cycle de vie du conteneur.
- ✓ **Ajouter un healthcheck** — Permettre à la plateforme de détecter un service non fonctionnel.
- ✓ **Versionner les images** — Préférer des tags de version explicites pour les déploiements reproductibles.
- ✓ **Un processus principal** — Garder chaque conteneur centré sur une responsabilité principale.

11. Résumé visuel

Dockerfile	La recette de construction.
docker build	Transforme la recette et le contexte en image.
Image	Modèle immuable et versionné.
docker run	Crée une instance à partir de l'image.
Conteneur	Processus isolé qui exécute l'application.
Volume	Stockage persistant indépendant du conteneur.
Compose	Décrit et démarre plusieurs services ensemble.

12. Sources officielles pour approfondir

- [Docker — What is an image?](#)
- [Docker — What is a container?](#)
- [Dockerfile reference](#)
- [Docker — Writing a Dockerfile](#)
- [Docker — Build context](#)